

NotebookForge

Hệ thống Tự động Sinh và Kiểm định Chất lượng Notebook
Thực hành Học máy bằng Hệ thống Đa tác nhân sử dụng
CrewAI

Nguyễn Xuân Huy^{*a},
Nguyễn Xuân Hoàng^a, Nguyễn Hoàng Nam^a,
Bùi Phong Hợp^a, Nguyễn Minh Trí^a

^a Nhóm 13: UTÚ

*Email nhóm trưởng: author@mail.com

1. Đặt vấn đề và giải pháp
2. Multi-agent Pipeline
3. Demo và Evaluation
4. Conclusion và Mentor Q&A
5. Conclusions
6. References

Contact Information

Address: 403.1 BK.B6, HCMUT Campus 2

Email:

mlandiotlab@gmail.com

Website: mliotlab.com

Facebook: facebook.com/hcmut.ml.iot.lab

YouTube: youtube.com/@mliotlab

Tổng quan dự án & bối cảnh thực tế

- ▶ **Tên đề tài:** NotebookForge - Hệ thống Tự động Sinh và Kiểm định Chất lượng Notebook Thực hành Học máy bằng Hệ thống Đa tác nhân.
- ▶ **Thách thức của người học AI/ML:** Người mới bắt đầu gặp khó khăn khi tìm kiếm bài tập thực hành cá nhân hóa theo đúng trình độ, thời lượng rảnh và mục tiêu cá nhân; tự học dễ nản vì thiếu lời giải chi tiết.
- ▶ **Hạn chế của LLM hiện tại:** Notebook do ChatGPT/Claude thông thường sinh ra hay bị lỗi không chạy được (runtime error), dễ bị rò rỉ dữ liệu (data leakage), thiếu tính cấu trúc sự phạm, hoặc tự bị đặt thông tin lý thuyết không chuẩn xác (hallucination).

Literature Review Highlights

- ▶ Author et al. (Year): Key finding 1.[?]
- ▶ Author et al. (Year): Key finding 2.[?]

Research Gap: *Describe gap identified.*

Giải pháp **NotebookForge** được đặt ra với 3 mục tiêu chính:

❶ Cá nhân hóa lộ trình (Personalization)

- ▶ Không sinh các bài học chung chung.
- ▶ Hệ thống đánh giá năng lực thực tế qua bộ Quiz 5 câu để xác định chính xác trình độ (`level_final`) và dung lượng bài dựa theo thời lượng học do người dùng đăng ký (`duration_minutes`).

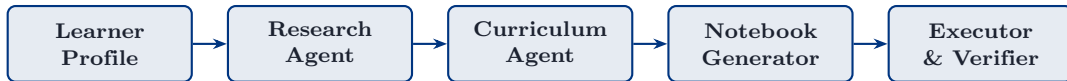
❷ Tự thực thi & sửa lỗi (Sandbox & Self-correction)

- ▶ Code không chỉ nằm trên giấy. Tất cả notebook được đưa vào môi trường cách ly (Sandbox) để chạy thử (Run-all).
- ▶ Nếu phát hiện lỗi Python, agent sẽ đọc vết lỗi (Traceback log) và tự sửa code (Retry Loop) đến khi chạy thành công 100%.

❸ Tri thức neo giữ & Kiểm định kép (Grounding & Dual-Verification)

- ▶ Nội dung lý thuyết được neo giữ vào Knowledge Base chuẩn (Scikit-Learn User Guide) để chống bịa đặt.
- ▶ Đầu ra bắt buộc phải vượt qua bộ kiểm định kép: 8 Hard Rules (luật cứng kỹ thuật) và LLM Judge (chấm điểm chất lượng sư phạm).

Sơ đồ pipeline tổng quan:



- ▶ Mỗi Agent đóng vai trò như một công nhân chuyên môn hóa trong dây chuyền sản xuất notebook tự động với các nhiệm vụ riêng biệt.

Thu thập yêu cầu & đánh giá trình độ (Frontend Layer)

❶ Form cấu hình

Người dùng chọn:

- ▶ Topic (Logistic Regression, Decision Tree, K-Means Clustering)
- ▶ Level tự đánh giá (Beginner / Intermediate)
- ▶ Thời lượng bài học (`duration_minutes`: 60–120 phút)
- ▶ Số lượng bài tập (1–5)

để xác định `level_final` và `duration_minutes`.

❷ Quiz 5 câu cho từng topic

- ▶ Căn chỉnh trình độ thực tế.
- ▶ Nếu người dùng chọn "Intermediate" nhưng làm sai nhiều câu Quiz, hệ thống tự động hạ chỉ số `level_final` về đúng năng lực thực tế để tránh bài học quá sức.

❸ Cơ chế UI Fallback

Giao diện tự phục hồi và hiển thị thông báo trạng thái mượt mà nếu Server backend xử lý chậm hoặc gặp sự cố mạng.

Research Agent: Quy trình RAG & Đóng gói ResearchBundle

- ▶ **Knowledge Base (.md):** Kho tri thức chuẩn hóa từ tài liệu chính thống (Scikit-Learn User Guide), đóng vai trò "sách giáo khoa" tham chiếu chống bịa đặt (hallucination).
- ▶ **Kiến trúc RAG 2 giai đoạn (Two-Stage RAG):**
 - ▷ **Giai đoạn 1 (BM25 Retrieval):** Truy xuất chính xác các đoạn văn bản lý thuyết, công thức và code mẫu liên quan từ Knowledge Base.
 - ▷ **Giai đoạn 2 (LLM Re-ranking):** Tự động sắp xếp, lọc bớt khái niệm dựa trên thời lượng `duration_minutes` và trình độ `level_final`.
- ▶ **Fallback Web Search (Wikipedia API):** Khi Knowledge Base không phủ hết khái niệm, hệ thống gọi Wikipedia API với bộ lọc token hit ratio ≥ 0.5 để lấy thông tin bổ trợ.
- ▶ **Đầu ra - ResearchBundle:** Tổng hợp nội dung đã RAG (Lý thuyết, Code mẫu, Mẹo thực hành) thành một gói thông tin cấu trúc để cung cấp trực tiếp cho Curriculum Agent.

Curriculum & Generator Agent: Sinh lộ trình học & Đóng gói Notebook

- ▶ **Curriculum Agent:** Sinh lộ trình học dựa trên **ResearchBundle**, phù hợp riêng cho từng dạng: Học có giám sát (Supervised) hoặc Học không có giám sát (Unsupervised).
- ▶ **Cấu trúc Notebook chuẩn 4 khối:**
 1. **Tổng quan bài học:** Giới thiệu về Topic và dataset (Tên, Nguồn, Kích thước, Biến mục tiêu, Đặc tả)
 2. **Chuẩn bị dữ liệu:** Nạp dữ liệu với bộ dataset tương ứng (Heart Failure, Red Wine Quality, Mall Customers) và chèn sẵn code preprocessing, EDA, split train/test (Supervised).
 3. **Nội dung & Bài tập thực hành:** Giới thiệu kiến thức nền tảng, code demo giải mẫu bài tập, nêu các yêu cầu bài tập dựa trên lộ trình học.
 4. **Kiểm tra tự động:** Chèn sẵn các đoạn code dạng **assert** để người học tự kiểm tra đáp án.

Executor, Verifier & Vòng lặp Tự sửa lỗi

- ▶ **Executor Sandbox:** Môi trường chạy code ảo cách ly hoàn toàn, nhận file .ipynb và thực thi từ trên xuống dưới để bắt chính xác các lỗi runtime.
- ▶ **Bộ kiểm định kép (Verifier):**
 - ▷ **8 Hard Rules (Quy tắc cứng):** Kiểm tra lỗi kỹ thuật bằng code (0 Execution Error, đúng định dạng JSON Schema, không Data Leakage, chứa đủ cell Todo/Assert...).
 - ▷ **LLM Judge (Chấm điểm mềm):** Đánh giá chất lượng bài học trên 4 tiêu chí sơ phạm (Khả năng thực thi, Độ chuẩn xác lý thuyết, Độ phù hợp trình độ, Tính logic sơ phạm).
- ▶ **Cơ chế Retry Loop:** Khi Verifier phát hiện lỗi, hệ thống không bỏ cuộc mà trích xuất vết lỗi dạng [CELL n] ... FIX: ... đưa lại cho Generator để sửa đúng ô code bị lỗi (tối đa 3 lần thử).

Minh họa vận hành hệ thống

- ▶ Quá trình tương tác trên giao diện Frontend: Chọn topic, thực hiện bài Quiz căn chỉnh trình độ.
- ▶ Minh họa quá trình sinh lộ trình tự động qua các Agent.
- ▶ Trực quan hóa vòng lặp tự phát hiện lỗi runtime và tự động sửa code (Retry Loop) trong Executor Sandbox.
- ▶ Sản phẩm đầu ra hoàn chỉnh: Notebook Jupyter đạt 100% khả năng thực thi.

Kết quả Kỹ thuật, Benchmark Metrics & Ablation Study

- ▶ **Thử nghiệm trên 6 Test Cases tiêu biểu:** Nhằm tối ưu hóa chi phí API mà vẫn đảm bảo tính đại diện, 6 bài thử nghiệm được thiết kế phủ đều 3 chủ đề (Logistic Regression, Decision Tree, K-Means) trên 2 cấp độ trình độ khác nhau.
- ▶ **Chỉ số đánh giá chính (Evaluation Metrics):**
 - ▷ **Execution Pass Rate:** Tỷ lệ Notebook chạy thành công 100% từ trên xuống dưới, không gặp bất kỳ lỗi Python runtime nào.
 - ▷ **Final Pass Rate:** Tỷ lệ Notebook vượt qua cả 8 Hard Rules kỹ thuật VÀ đạt điểm LLM Judge $\geq 4/5$.
 - ▷ **LLM Judge Agreement:** Độ đồng thuận cao với chuyên gia human benchmark ($\kappa \geq 0.70$).
- ▶ **Kết quả Ablation Study (Đánh giá vai trò từng thành phần):**
 - ▷ **Thiếu RAG ResearchBundle:** Tỷ lệ bịa đặt kiến thức (hallucination) tăng, lý thuyết rời rạc không bám sát chuẩn Scikit-Learn.
 - ▷ **Thiếu Retry Loop (Sandbox):** Tỷ lệ Execution Pass Rate giảm mạnh do không khắc phục được các lỗi runtime nhỏ phát sinh lúc sinh code.
 - ▷ **Thiếu Hard Rules / LLM Judge:** Notebook dễ vi phạm lỗi Data Leakage hoặc thiếu cấu trúc bài tập chuẩn sự phạm.

Tóm tắt đóng góp

- ▶ Đề xuất kiến trúc Multi-agent tự động sinh Notebook thực hành ML cá nhân hóa.
- ▶ Tích hợp cơ chế kiểm định kép (Hard Rules + LLM Judge) đảm bảo độ chính xác kỹ thuật và giá trị sư phạm.
- ▶ Xây dựng vòng lặp tự sửa lỗi (Retry Loop) trong môi trường Sandbox giúp duy trì tỷ lệ thực thi code thành công cao.

Kế hoạch hoàn thiện & Q&A

- ▶ Mở rộng Knowledge Base sang các chủ đề Deep Learning và Computer Vision.
- ▶ Tối ưu hóa thời gian sinh bài và chi phí gọi LLM API.
- ▶ Thử nghiệm thực tế với nhóm người dùng sinh viên để thu thập feedback.

Thank You

Acknowledgements

The authors gratefully acknowledge Machine Learning & IoT Lab, HCMUT for their support and guidance.